

Chapter 3:

Data Visualization



Data visualization

- The representation of data through use of common graphics, such as charts, plots, infographics, and even animations.
- These visual displays of information communicate complex data relationships and data-driven insights in a way that is easy to understand.

Outlines

- General Matplotlib Tips
- Simple Line Plots
- Simple Scatter Plots
- Box plot
- Visualizing Errors
- Density and Contour Plots
- Histograms, Binnings, and Density

General Matplotlib Tips

General Matplotlib Tips

- A fundamental part of the data scientist's toolkit is data visualization.
- Matplotlib tool for visualization in Python
- Importing matplotlib

```
import matplotlib.pyplot as plt
```

<https://seaborn.pydata.org/>

General Matplotlib Tips

show() or No show()? How to Display Your Plots

How you view your Matplotlib plots depends on the context

Plotting from a script

----- file: myplot.py -----

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
plt.plot(x, np.sin(x))
```

```
plt.plot(x, np.cos(x))
```

```
plt.show()
```

General Matplotlib Tips

show() or No show()? How to Display Your Plots

How you view your Matplotlib plots depends on the context

Plotting from an IPython notebook

```
# ----- file: myplot.py -----
```

```
%matplotlib inline
```

```
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
fig = plt.figure()
```

```
plt.plot(x, np.sin(x), '-')
```

```
plt.plot(x, np.cos(x), '--');
```

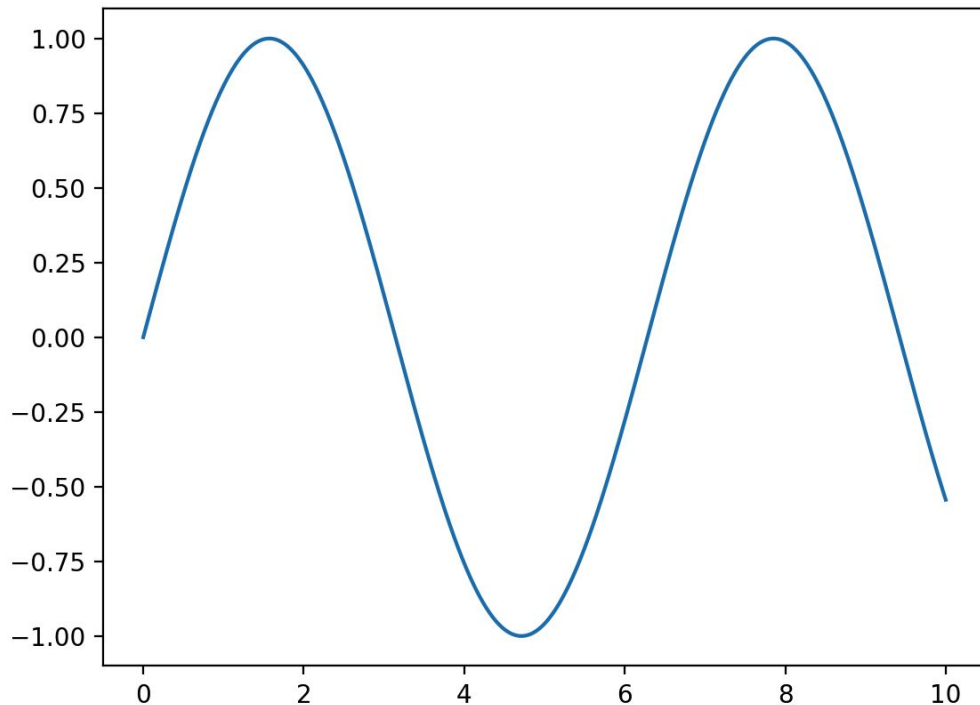
Simple Line Plots

Line charts are a good choice for showing trend.

Simple Line Plots

Creating a simple plot of a single function $y = f(x)$

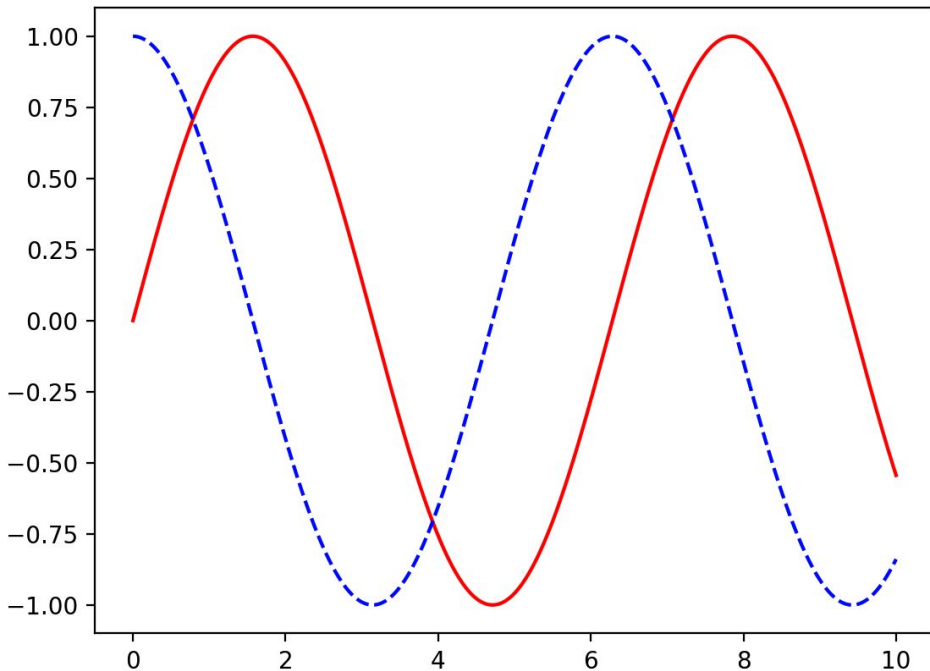
```
import matplotlib.pyplot as plt
import numpy as np
fig = plt.figure()
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x))
plt.show()
```



Simple Line Plots

Adjusting the Plot: Line Colors and Styles

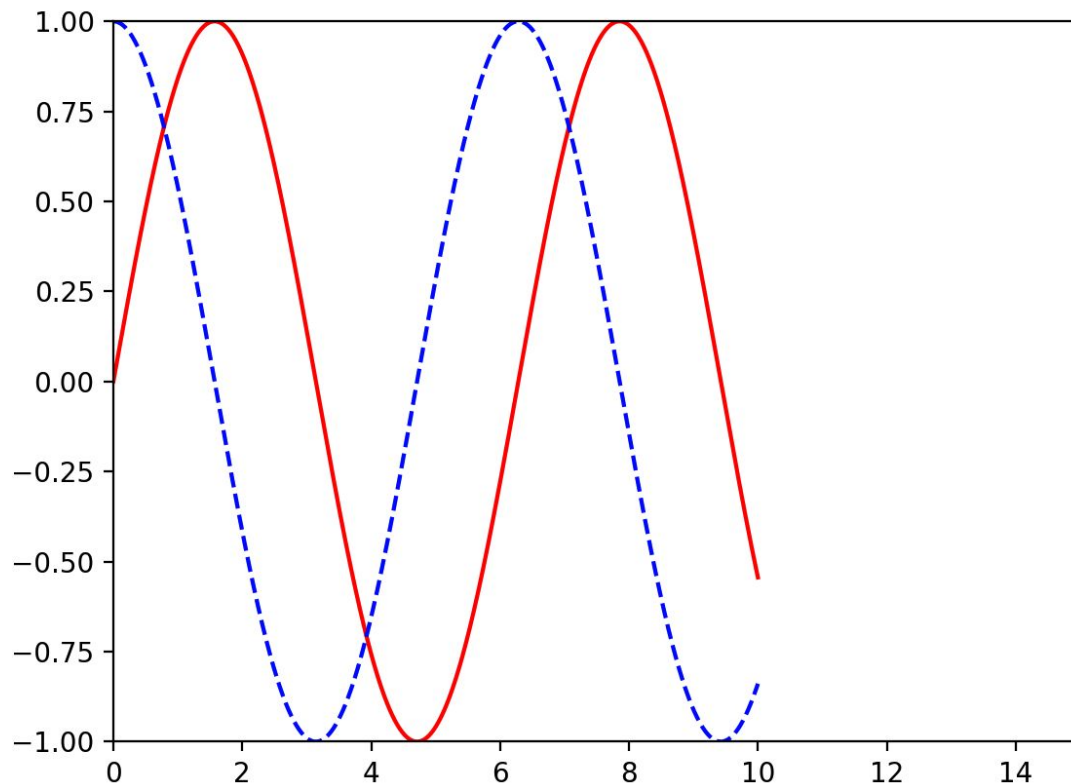
```
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x),
         color="red", linestyle='solid')
plt.plot(x, np.cos(x),
         color="blue", linestyle='dashed')
plt.show()
```



Simple Line Plots

Adjusting the Plot: Axes Limits

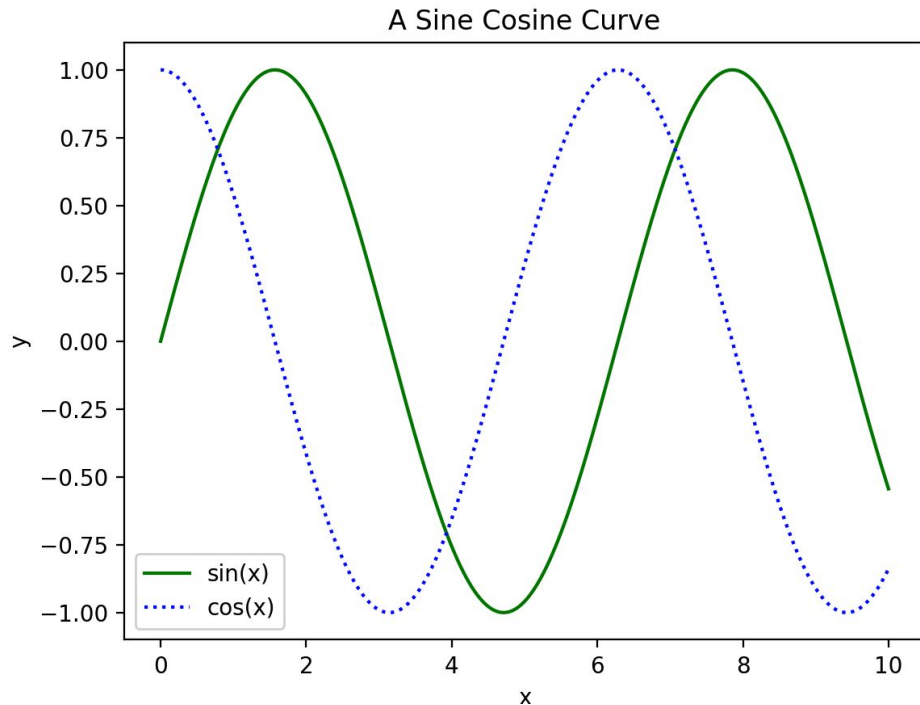
```
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x),
         color="red", linestyle='solid')
plt.plot(x, np.cos(x),
         color="blue", linestyle='dashed')
plt.xlim(0, 15)
plt.ylim(-1, 1)
plt.show()
```



Simple Line Plots

Labeling Plots

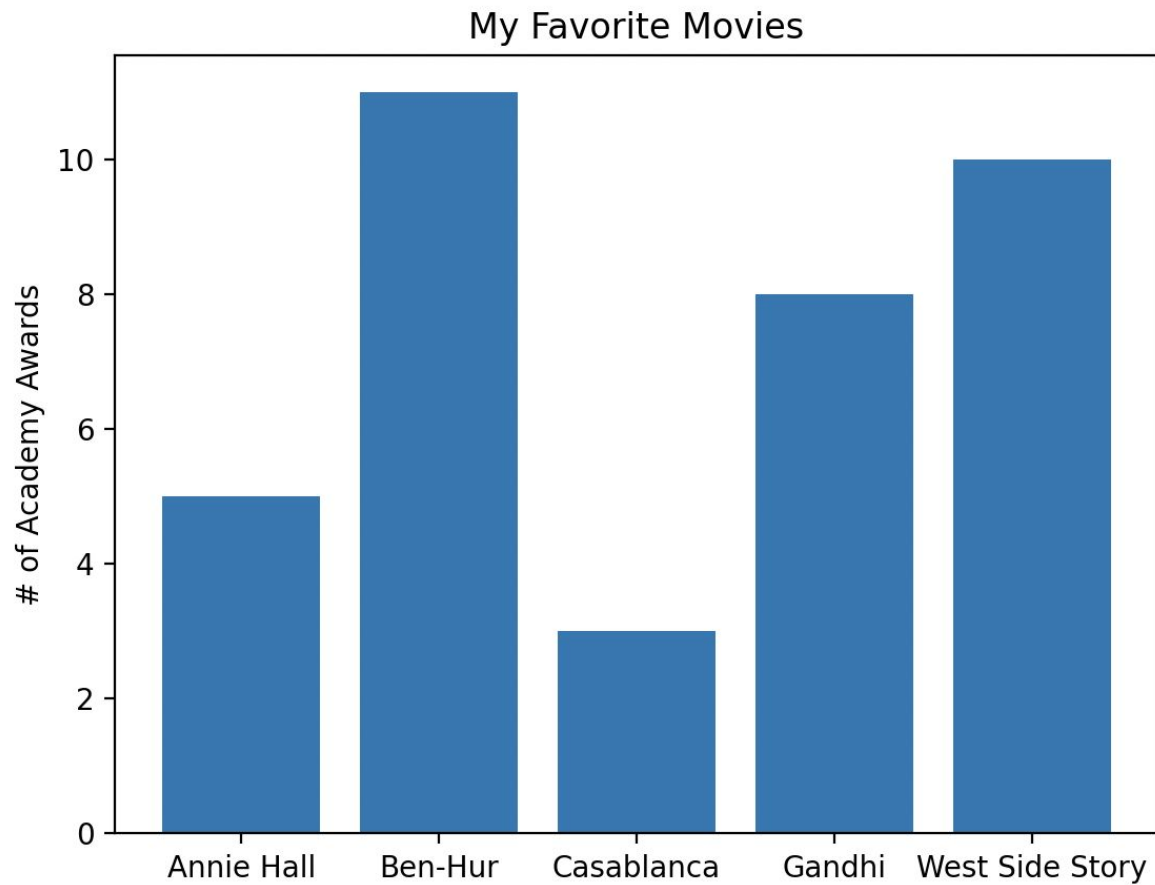
```
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), '-g',
         label='sin(x)')
plt.plot(x, np.cos(x), 'b:',
         label='cos(x)')
plt.title("A Sine Cosine Curve")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```



Bar Charts

A bar chart is a good choice when you want to show how some quantity varies among some discrete set of items.

Bar Charts



Bar Charts

```
import matplotlib.pyplot as plt

movies = ["Annie Hall", "Ben-Hur", "Casablanca", "Gandhi", "West Side
Story"]

num_oscars = [5, 11, 3, 8, 10]

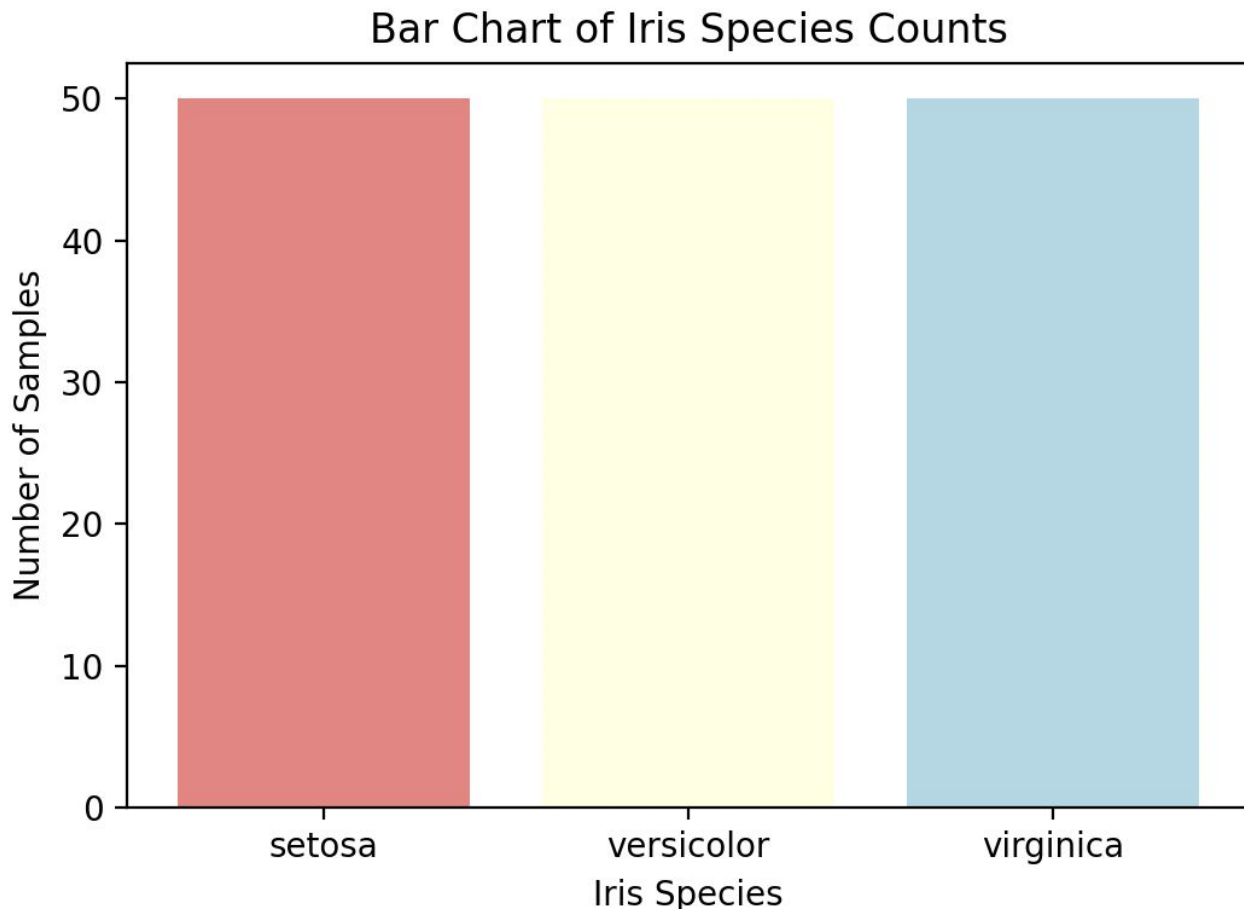
plt.bar(movies, num_oscars)

plt.ylabel("# of Academy Awards")

plt.title("My Favorite Movies")

plt.show()
```

Bar Charts - Iris dataset sklearn



Bar Charts - Iris dataset sklearn

```
import matplotlib.pyplot as plt
from sklearn.datasets import
load_iris
import numpy as np
```

```
# Load Iris dataset
iris = load_iris()
y = iris.target
target_names = iris.target_names
```

```
# Count samples per species
counts = np.bincount(y)
```

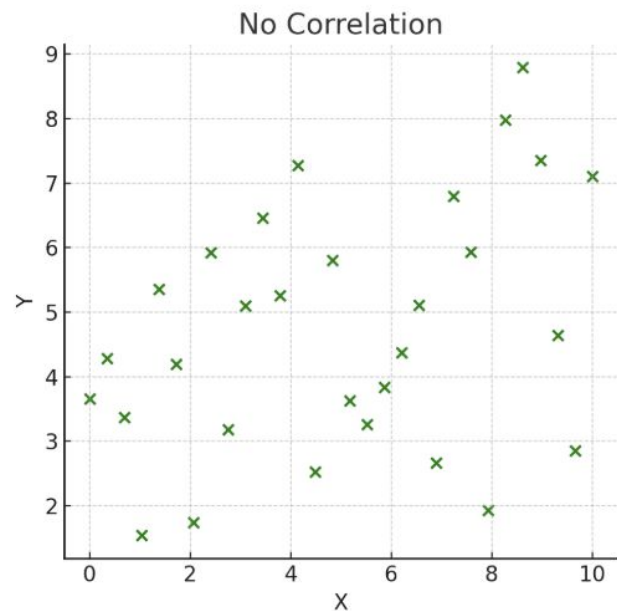
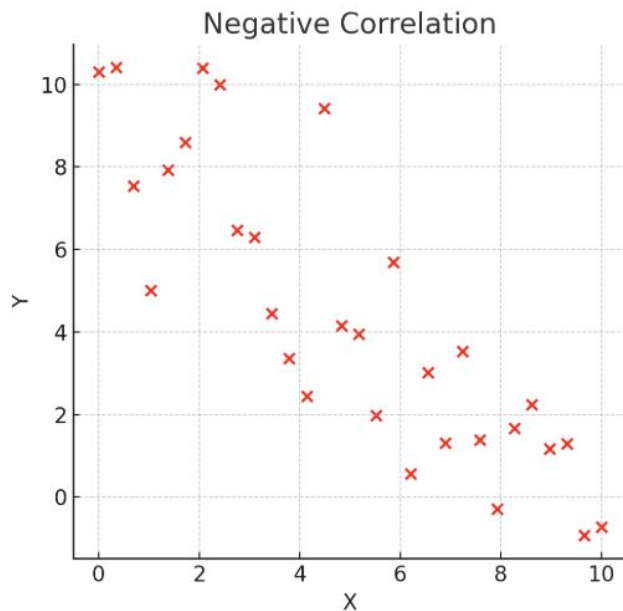
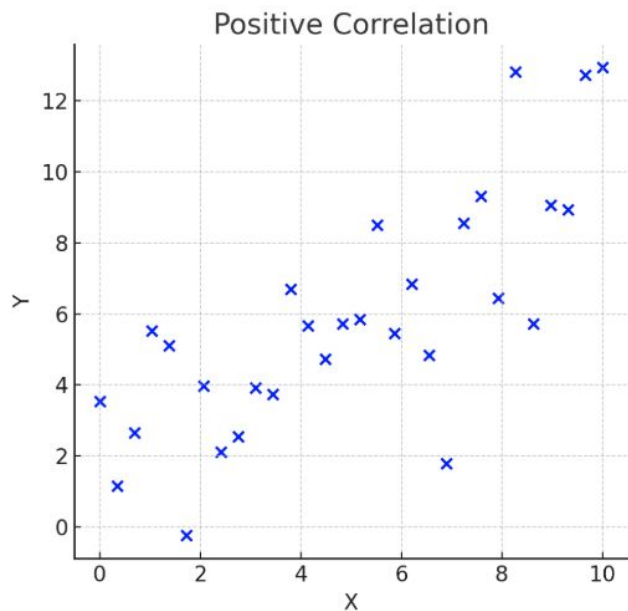
```
# Plot bar chart
plt.figure(figsize=(6, 4))
plt.bar(target_names, counts,
color=['lightcoral',
'lightyellow', 'lightblue'])
plt.xlabel("Iris Species")
plt.ylabel("Number of Samples")
plt.title("Bar Chart of Iris
Species Counts")
# plt.grid(axis='y', alpha=0.7)
plt.show()
```

Simple Scatter Plots

A scatterplot is the right choice for visualizing the relationship between two variables, detecting trends, clusters, or outliers.

Scatter Plots

- The x-axis represents the independent variable.
- The y-axis represents the dependent variable.
- Each point on the chart corresponds to a pair of values (x,y)



Scatter Plots with plt.plot

```
import matplotlib.pyplot as plt
```

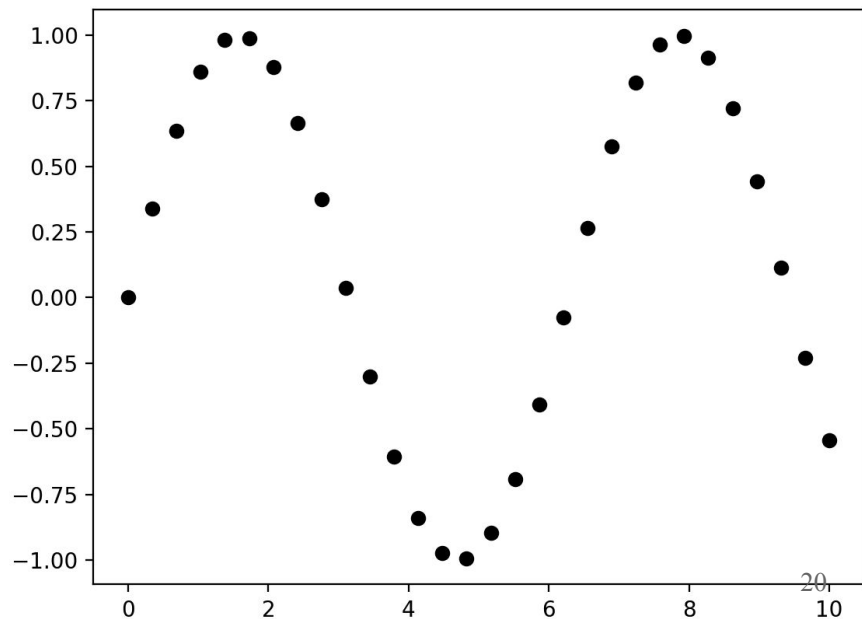
```
import numpy as np
```

```
x = np.linspace(0, 10, 30)
```

```
y = np.sin(x)
```

```
plt.plot(x, y, 'o', color='black')
```

```
plt.show()
```



Scatter Plots with plt.scatter

```
import matplotlib.pyplot as plt
```

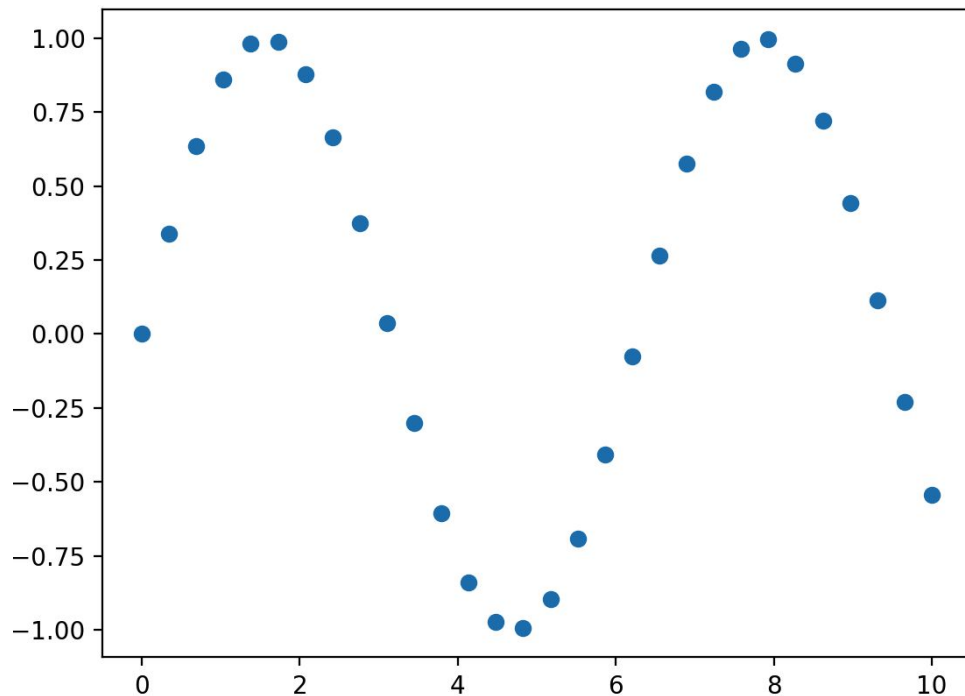
```
import numpy as np
```

```
x = np.linspace(0, 10, 30)
```

```
y = np.sin(x)
```

```
plt.scatter(x, y, marker='o')
```

```
plt.show()
```



Scatter Plots with plt.scatter

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
```

```
# Load Iris dataset
iris = load_iris()
X = iris.data[:, :2]  # only sepal length and sepal width
y = iris.target
target_names = iris.target_names
```

```
# Create scatter plot
plt.figure(figsize=(7, 5))
for i, target_name in enumerate(target_names):
    plt.scatter(
        X[y == i, 0],  # sepal length
        X[y == i, 1],  # sepal width
        label=target_name,
        alpha=0.7
```

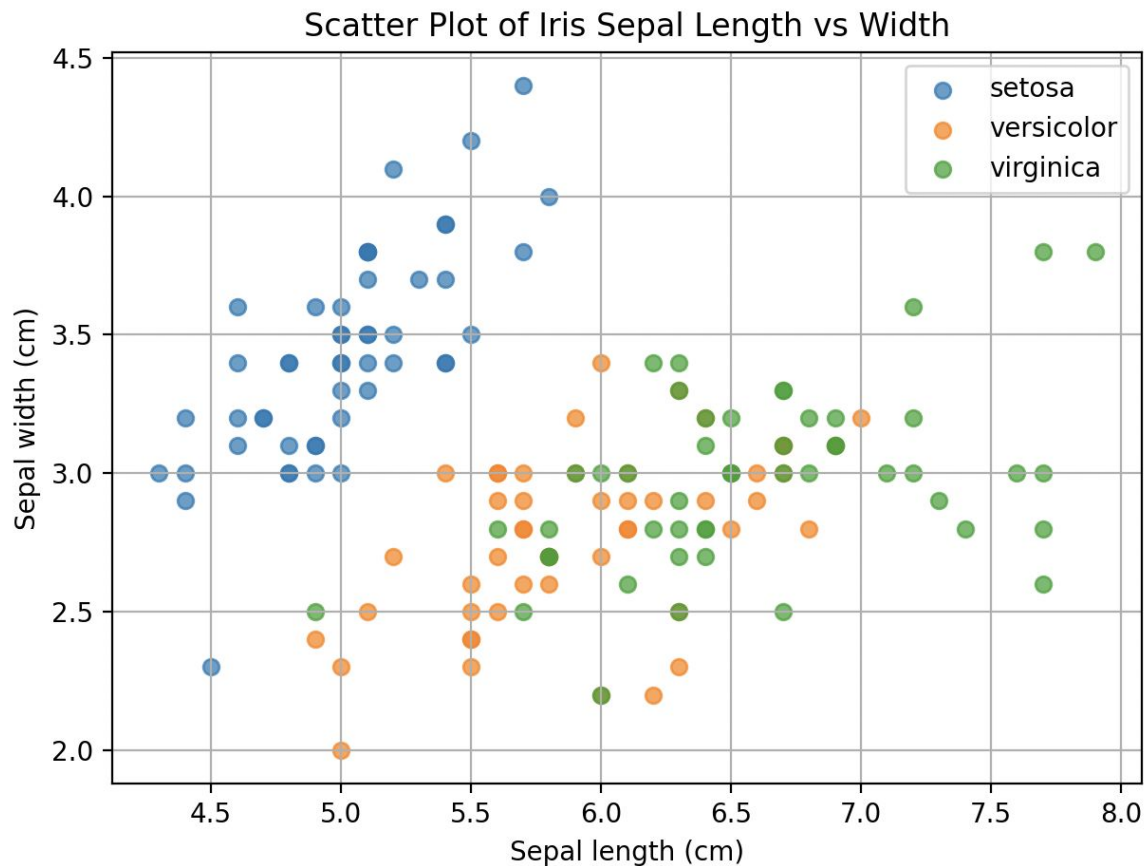
Explore 4 dimensions of the Iris data:

- (x, y) corresponds to the sepal length and width,
- the color is related to the particular species of flower.

```
plt.xlabel("Sepal length (cm)")
plt.ylabel("Sepal width (cm)")
plt.title("Scatter Plot of Iris Sepal Length vs
Width")
plt.legend()
plt.grid(True)
plt.show()
```

```
)
```

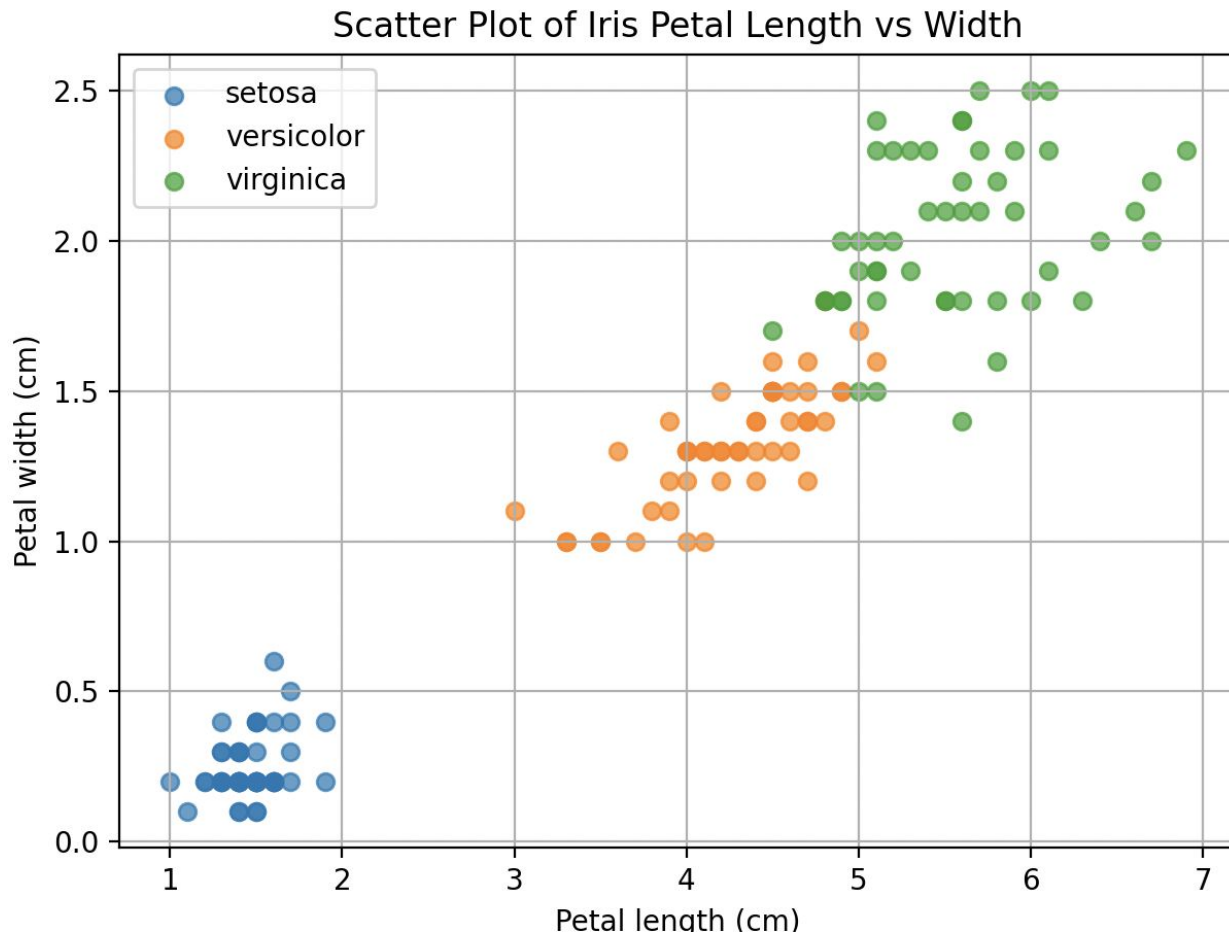
Scatter Plots with plt.scatter



Sepal length + width is enough to perfectly separate **Setosa** from the other two.

However, **Versicolor** and **Virginica** overlap heavily → these two species cannot be separated well using only sepal dimensions.

Scatter Plots with `plt.scatter`



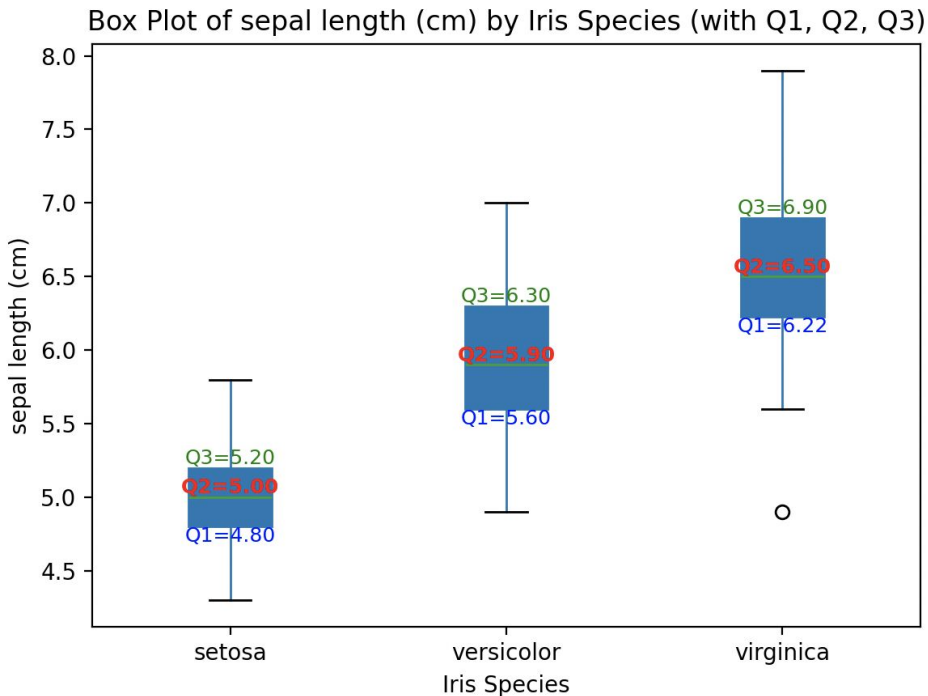
Box Plot

Box Plot displays distribution of a dataset

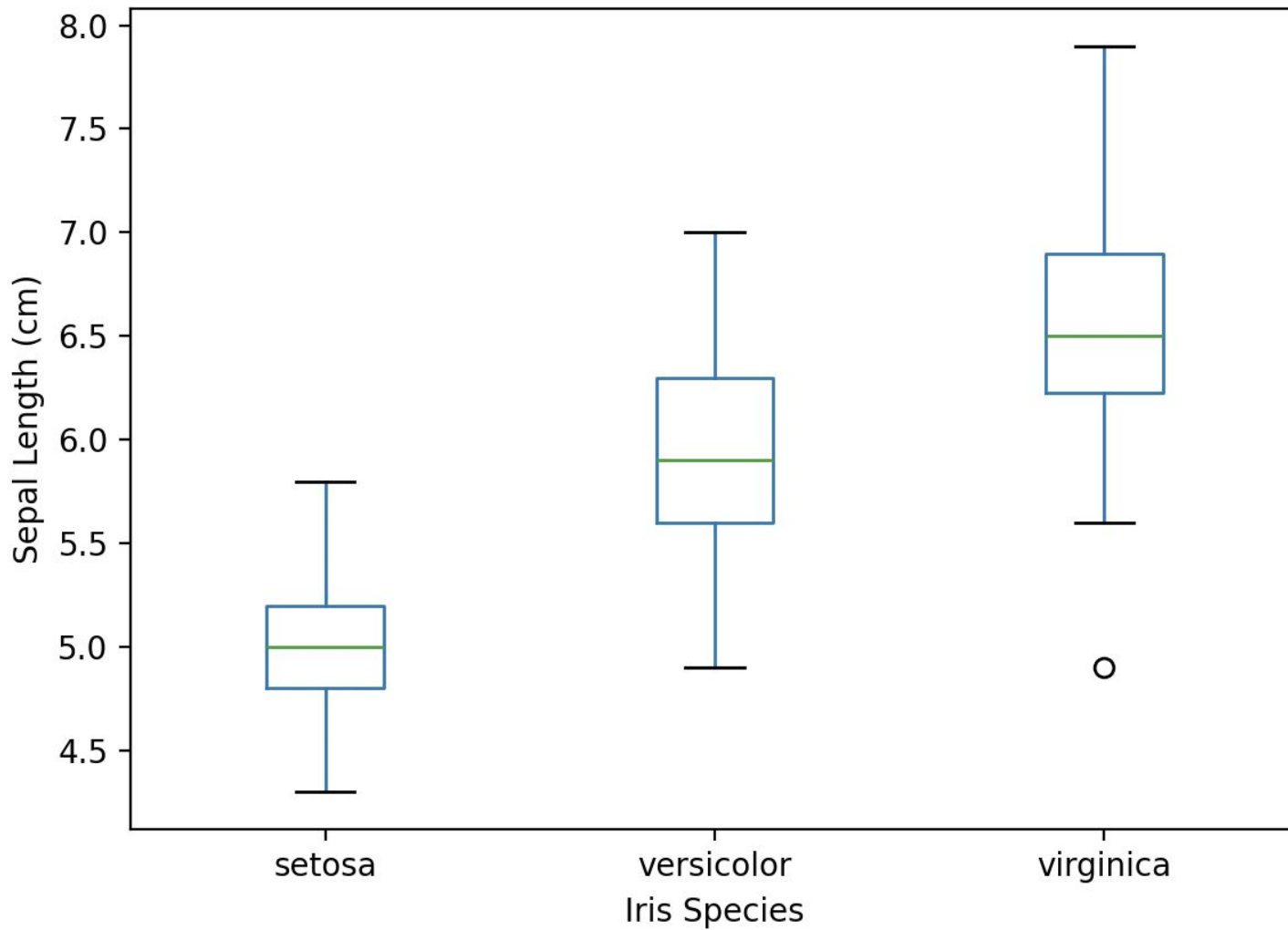
Box Plot

A typical boxplot comes with several components:

- Median (Q2): the line inside the box
- Quartiles (Q1 & Q3): the edges of the box
- Interquartile Range ($IQR = Q3 - Q1$)
box height/width
- Whiskers → data within $1.5 \times IQR$
- Outliers → points beyond the whiskers



Box Plot of Sepal Length by Iris Species



Box Plot

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd
```

```
# Load Iris dataset into a DataFrame
iris = load_iris(as_frame=True)
df = iris.frame # pandas DataFrame
```

```
# Create box plot for Sepal Length by Species
plt.figure(figsize=(7, 5))
df.boxplot(column="sepal length (cm)",
by="target", grid=False)
```

```
# Replace numeric target with species name
plt.xticks([1, 2, 3], iris.target_names)

plt.xlabel("Iris Species")
plt.ylabel("Sepal Length (cm)")
plt.title("Box Plot of Sepal Length by Iris
Species")
plt.show()
```

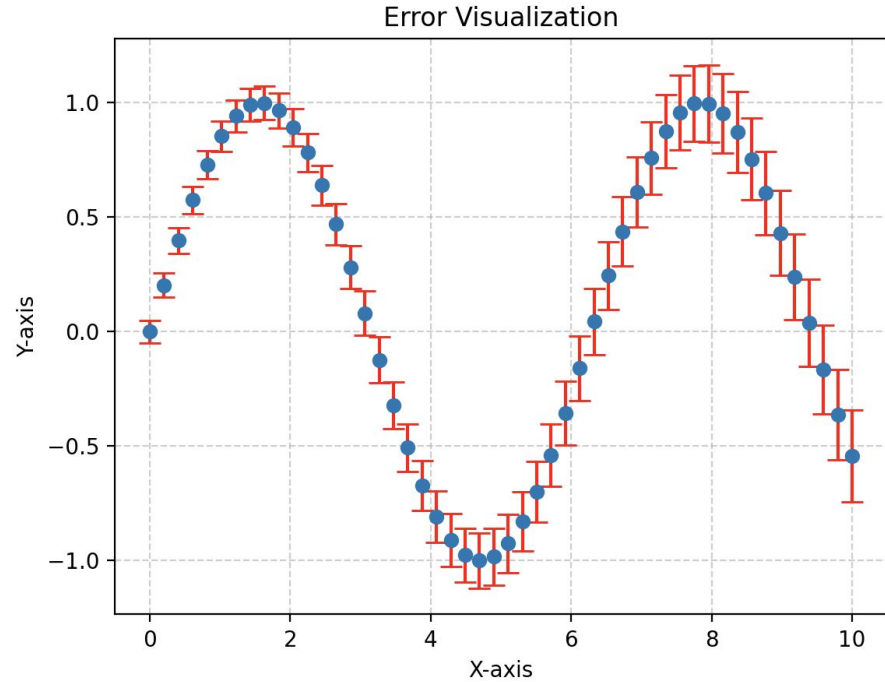
Visualizing Errors

Error bars

- Error bars show the uncertainty or variability in data.
- Commonly represent standard deviation, standard error, or confidence intervals.
- They are small vertical/horizontal lines extending from a plotted data point.

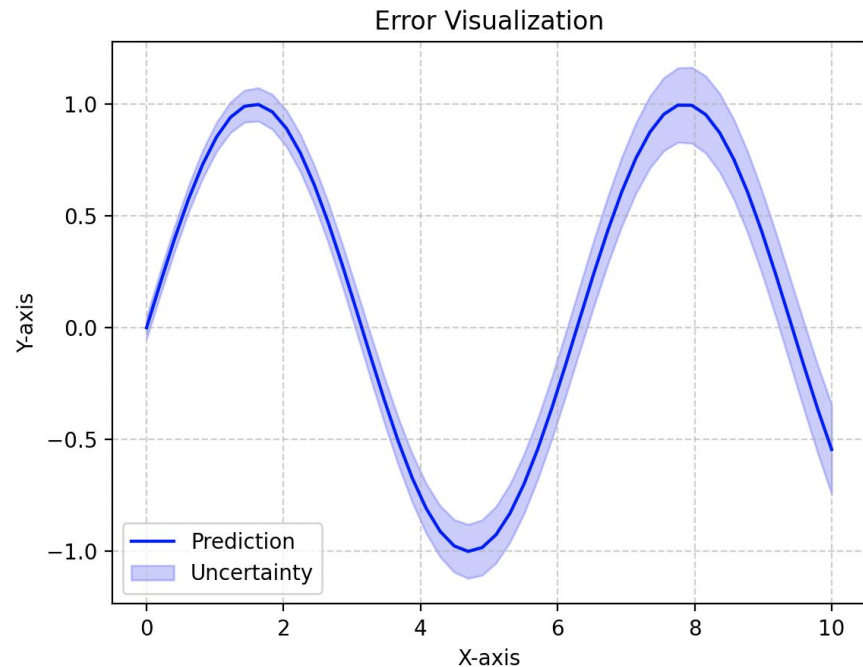
Basic Error bars

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, 50)
y = np.sin(x)
errors = np.linspace(0.05, 0.2, 50)
plt.errorbar(x, y, yerr=error, fmt='o',
             capsize=5,  ecolor='red')
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Error Visualization")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



Basic Error bars

Instead of separate error bars, we use a shaded region (as confidence interval) that covers all x-values.



Basic Error bars

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 50)
y = np.sin(x)
error = np.linspace(0.05, 0.2, 50)

# Upper and lower bounds
y_upper = y + error
y_lower = y - error

# Plot mean line
plt.plot(x, y, label="Prediction",
color="blue")
```

```
# Fill between lower and upper
(continuous error)
plt.fill_between(x, y_lower, y_upper,
color="blue", alpha=0.2,
label="Uncertainty")

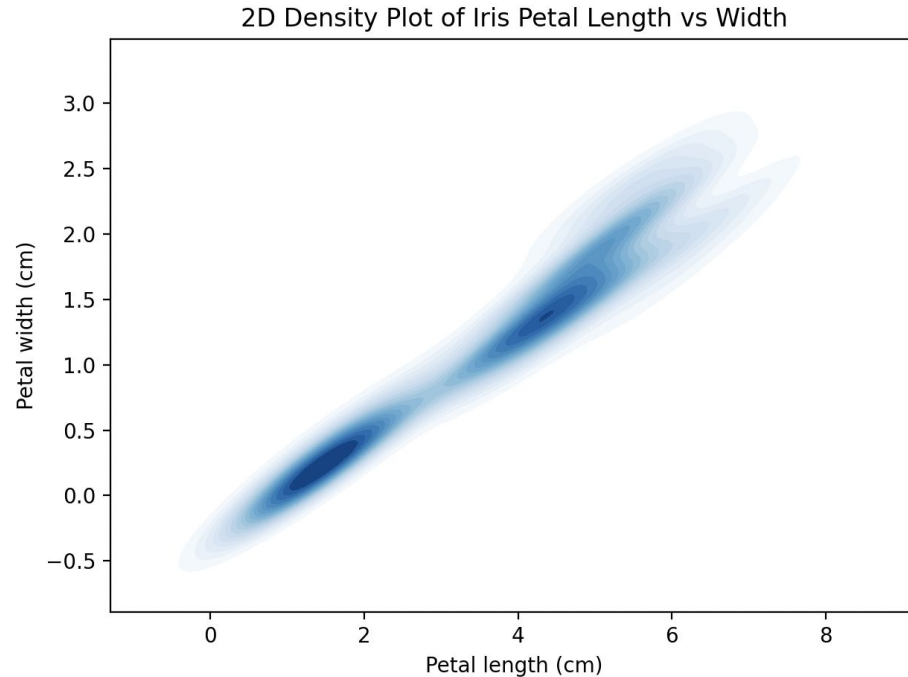
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Error Visualization")
plt.legend()
plt.grid(True, linestyle="--",
alpha=0.6)
plt.show()
```

Density and Contour Plots

Display three-dimensional data in two dimensions using contours or color-coded regions

Density Plot

- Shows the probability density of data in 2D.
- Darker (or more intense) regions = higher concentration of points.



Density Plot

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
import pandas as pd

# Load Iris dataset into a DataFrame
iris = load_iris(as_frame=True)
df = iris.frame
```

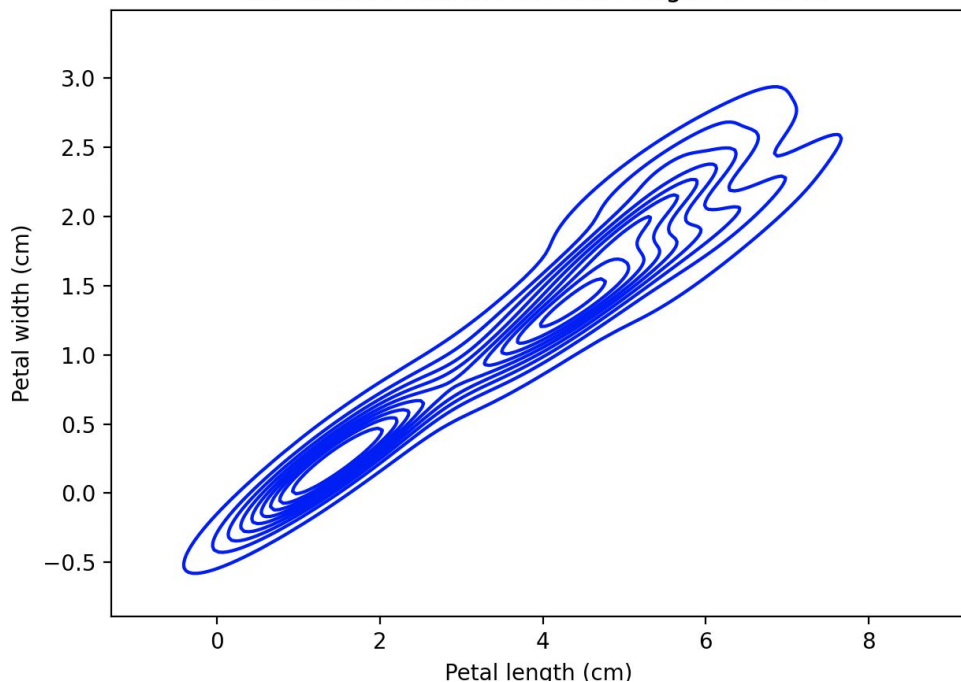
```
# Density plot (2D KDE)
plt.figure(figsize=(7, 5))
sns.kdeplot(
    data=df, x="petal length (cm)",
    y="petal width (cm)",
    fill=True, cmap="Blues",
    thresh=0.05, levels=20
)

plt.title("2D Density Plot of Iris
Petal Length vs Width")
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.show()
```

Contour Plot

- Similar to a density plot but uses contour lines instead of filled colors.
- Each contour line connects points of equal density (or value).

2D Contour Plot of Iris Petal Length vs Width

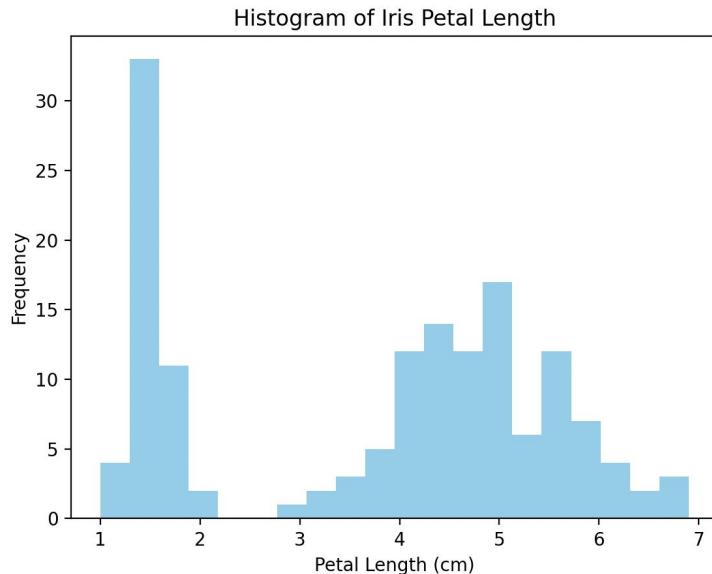


```
sns.kdeplot(data=df, x="petal length (cm)",  
            y="petal width (cm)",  
            levels=10, color="blue" # contour lines  
)
```

Histograms, Binnings, and Density

Histograms

- A histogram divides the data into intervals (bins) and counts how many observations fall into each bin.
- X-axis: the value ranges (bins).
- Y-axis: the count (frequency) or probability.



Histograms

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data[:, 2]  # petal length

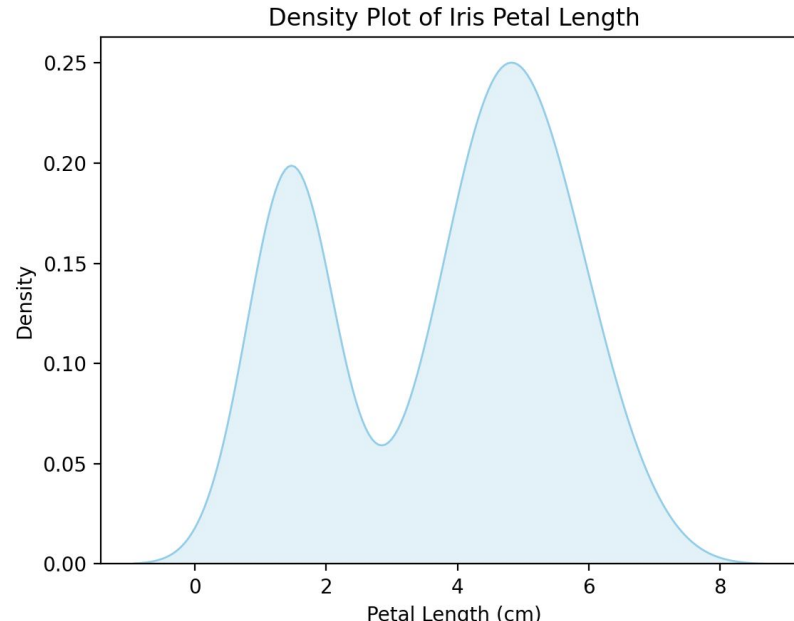
plt.hist(X, bins=20, color="skyblue", edgecolor="black")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Frequency")
plt.title("Histogram of Iris Petal Length")
plt.show()
```

Binning

- Binning is the process of splitting continuous data into discrete intervals (bins).
- A histogram is essentially a visualization of binning.
- The number of bins affects the visualization:
 - Too few bins → oversimplified, loss of detail.
 - Too many bins → too noisy, hard to see the trend.

Density

- Density is a smoothed version of a histogram.
- Instead of counting frequencies per bin, density estimates a continuous probability density function.



Density

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

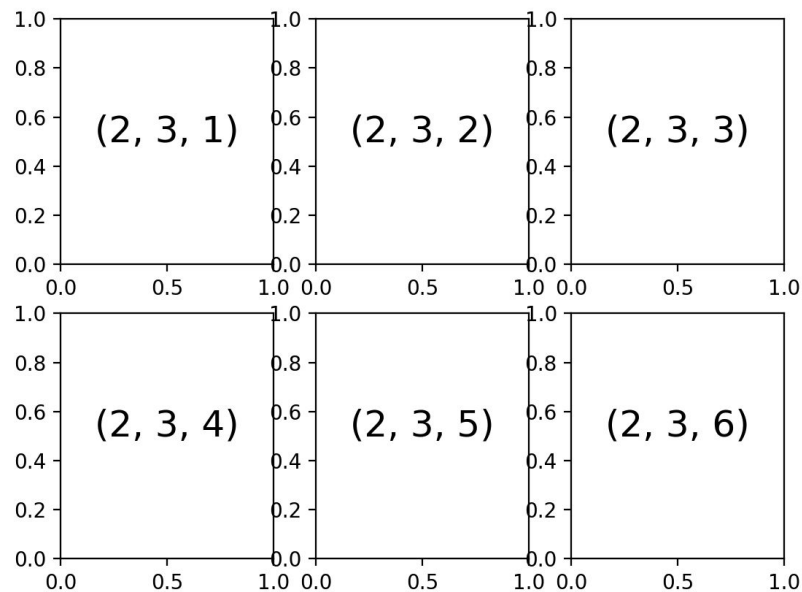
sns.kdeplot(df["petal length (cm)"], fill=True, color="skyblue")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Density")
plt.title("Density Plot of Iris Petal Length")
plt.show()
```

Multiple Subplots

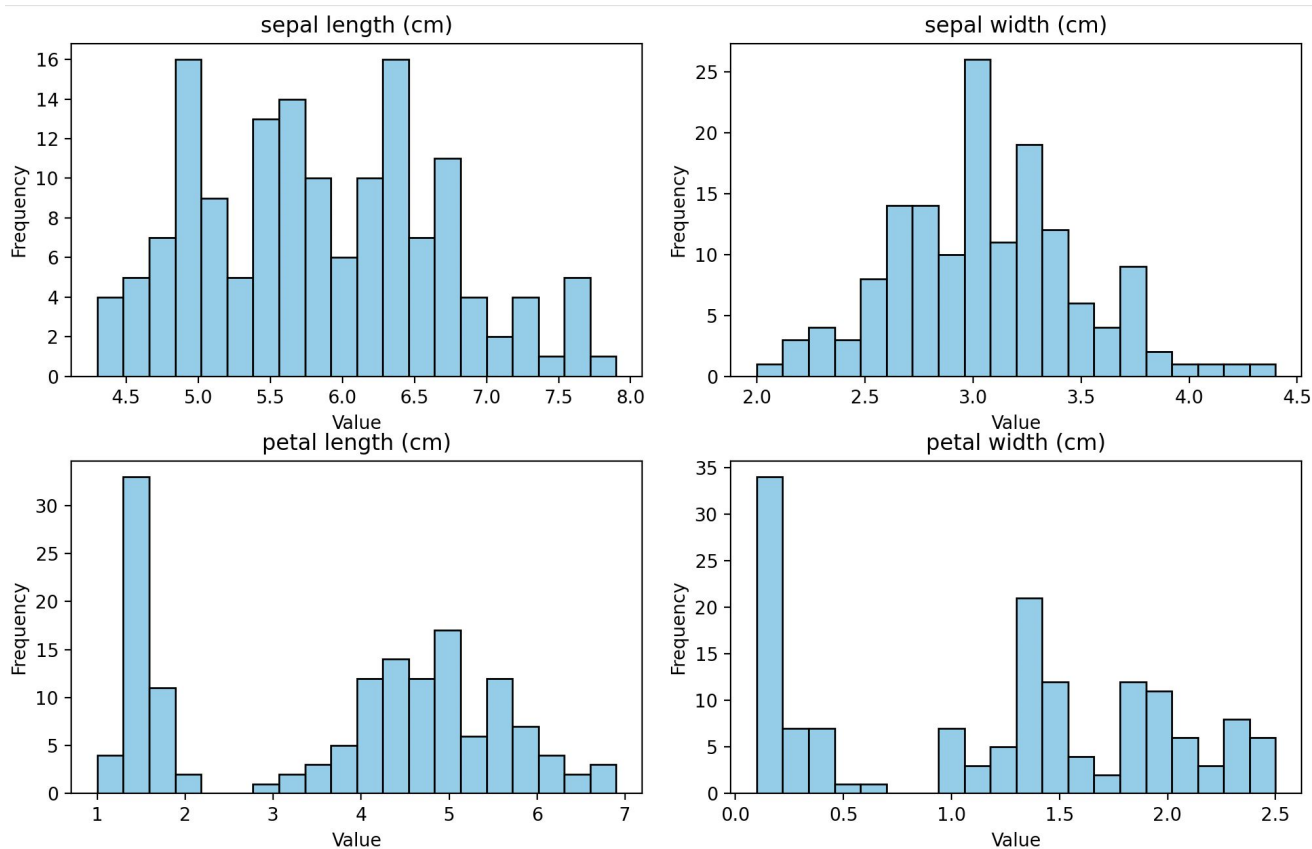
Multiple Subplots

- It is helpful to compare different views of data side by side.
- `plt.subplot()`, which creates a single subplot within a grid

```
import numpy as np
import matplotlib.pyplot as plt
for i in range(1, 7):
    plt.subplot(2, 3, i)
    plt.text(0.5, 0.5, str((2, 3, i)),
            fontsize=18, ha='center')
plt.show()
```



Multiple Subplots



Multiple Subplots

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris(as_frame=True)
df = iris.frame
features = df.columns[:4] # 4 features
print(features)

# Create a 2x2 grid of subplots
fig, axes = plt.subplots(2, 2,
figsize=(10, 8))
axes = axes.ravel() # flatten 2D array
of axes to 1D for easy iteration
```

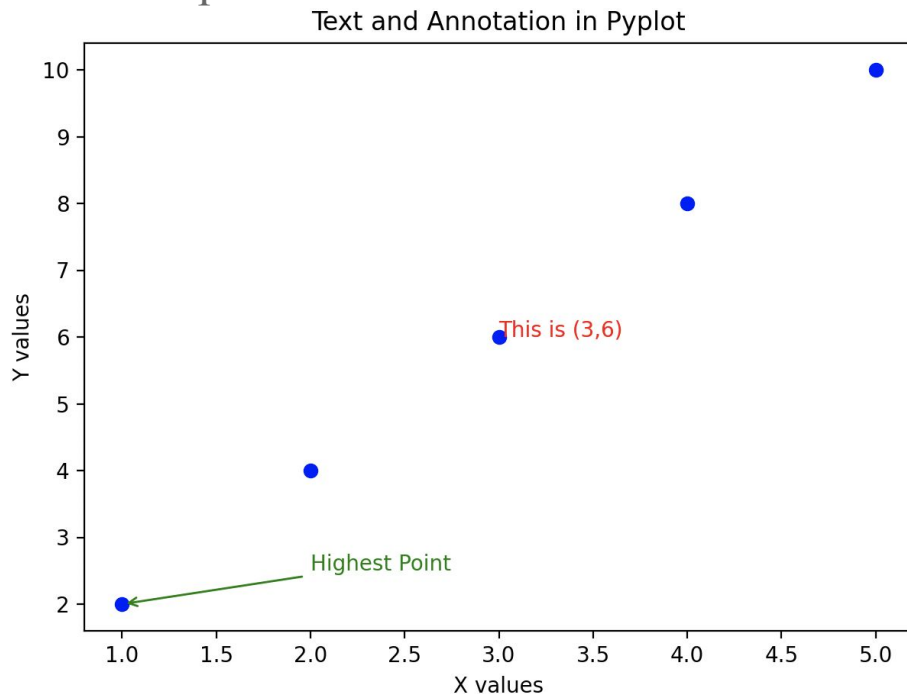
```
# Loop through features and draw
histogram
for i, feature in enumerate(features):
    print(i)
    axes[i].hist(df[feature], bins=20,
color="skyblue", edgecolor="black")
    axes[i].set_title(feature)
    axes[i].set_xlabel("Value")
    axes[i].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Text and Annotation

Text and Annotation

- `plt.text(x, y, "label")` → writes text directly at coordinates (x, y)
- `plt.annotate("text", xy=(.), xytext=(.), arrowprops=...)` → adds text with an arrow pointing to a chosen data point.



Text and Annotation

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
plt.figure(figsize=(7,5))
```

```
plt.scatter(x, y, color="blue")
```

```
# Add title and axis labels
```

```
plt.title("Text and Annotation in  
Pyplot")
```

```
plt.xlabel("X values")
```

```
plt.ylabel("Y values")
```

```
# Add simple text near a point
```

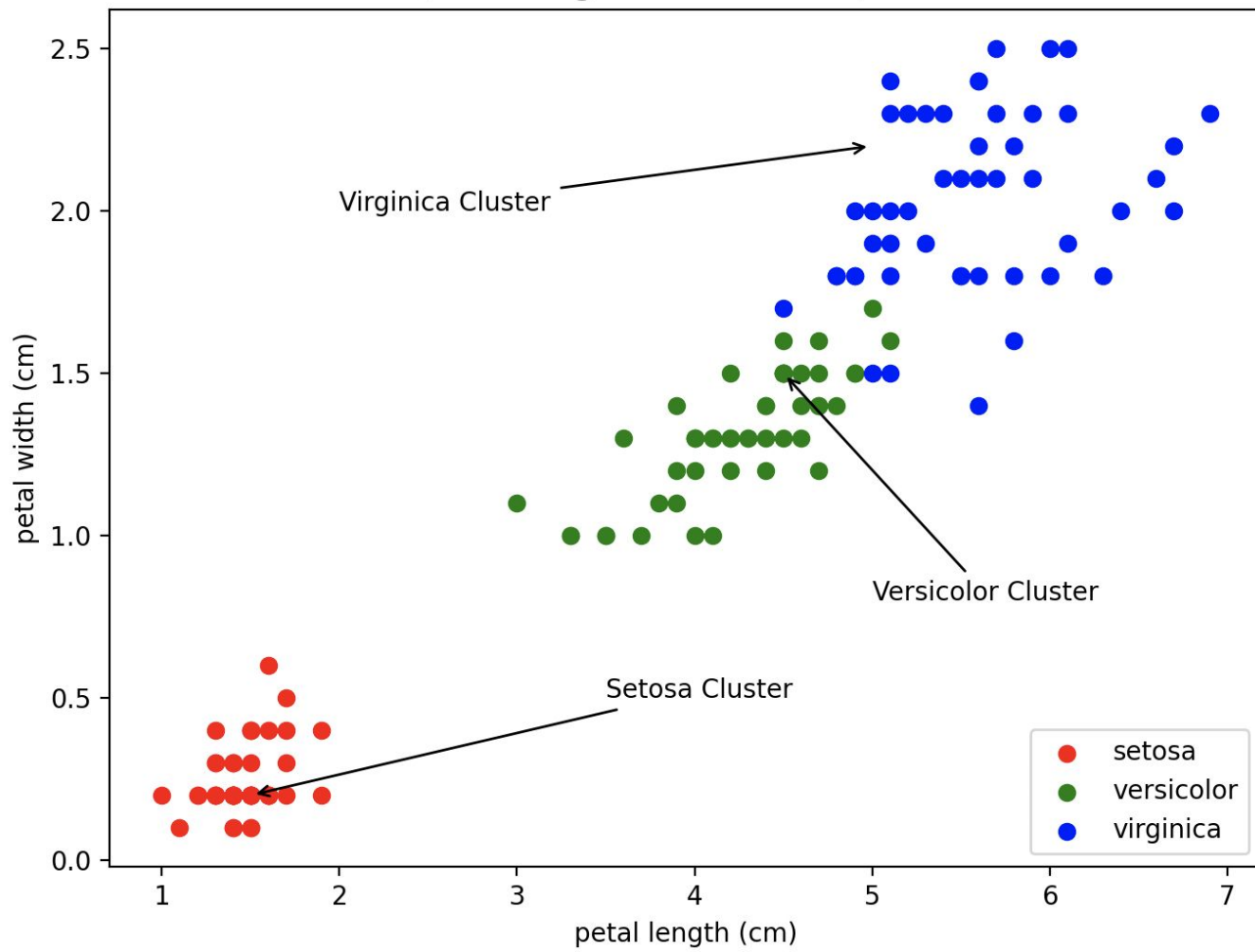
```
plt.text(3, 6, "This is (3,6)", fontsize=10,  
color="red")
```

```
# Add annotation with an arrow pointing to the  
highest point
```

```
plt.annotate("Highest Point",  
             xy=(1, 2), xycoords="data",      #  
             arrow points to this point  
             xytext=(2, 2.5), textcoords="data",  
             # text placed here  
             arrowprops=dict(arrowstyle="->",  
color="green"),  
             fontsize=10, color="green")
```

```
plt.show()
```

Iris Dataset (Petal Length vs Petal Width) with Annotations



Three-dimensional plotting

Three-dimensional plotting

Three-Dimensional Points and Lines

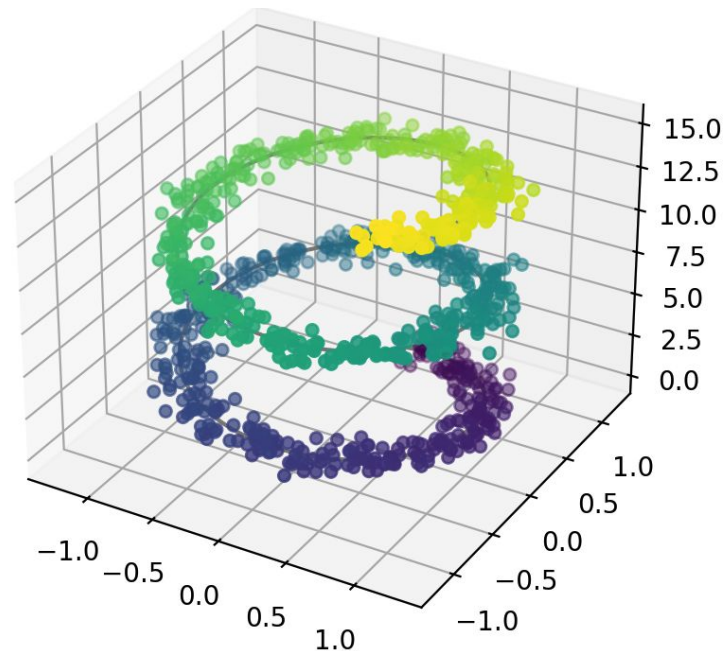
```
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = plt.axes(projection='3d')

zline = np.linspace(0, 15, 1000)
xline = np.sin(zline)
yline = np.cos(zline)
ax.plot3D(xline, yline, zline, 'gray')

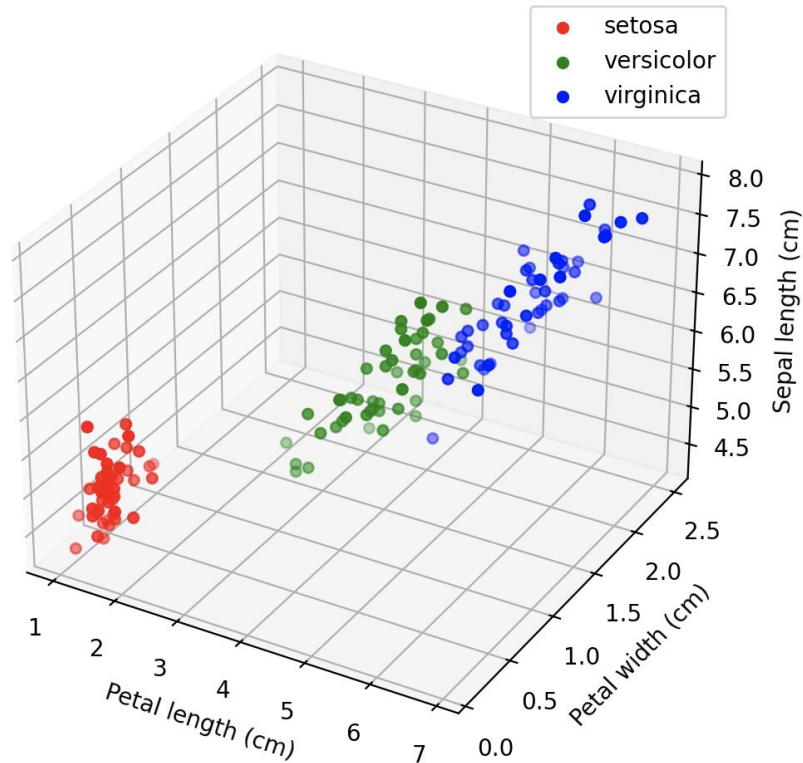
xdata = np.sin(zline) + 0.1 * np.random.randn(1000)
ydata = np.cos(zline) + 0.1 * np.random.randn(1000)
ax.scatter3D(xdata, ydata, zline, c=zline)

plt.show()
```



Three-dimensional plotting

3D Scatter Plot of Iris Dataset (Petal + Sepal)



Three-dimensional plotting

```
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load iris dataset
iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

# Create 3D scatter plot
fig = plt.figure(figsize=(8,6))
ax = plt.axes(projection='3d')

# Plot each species in different colors
colors = ['red', 'green', 'blue']
for target, color in zip(range(3), colors):
    ax.scatter3D(X[y==target, 2], # Petal length
                 X[y==target, 3], # Petal width
                 X[y==target, 0], # Sepal length
                 label=target_names[target], c=color)

# Axis labels
ax.set_xlabel("Petal length (cm)")
ax.set_ylabel("Petal width (cm)")
ax.set_zlabel("Sepal length (cm)")
ax.set_title("3D Scatter Plot of Iris Dataset (Petal + Sepal)")

ax.legend()
plt.show()
```